

Respostas dos Exercícios de Banco de Dados — Joins e Agregações

1- Exercício Proposto 1: INNER JOIN e autorrelacionamento

a) Liste pedidos com nome do cliente e status:

```
SELECT
    p.id_pedido,
    c.nome AS nome_cliente,
    p.status
FROM pedido p
INNER JOIN cliente c ON p.id_cliente = c.id_cliente;
```

b) Liste os produtos e subtotais de cada item:

```
SELECT
    p.nome AS nome_produto,
    ip.quantidade,
    ip.preco_unitario,
    (ip.quantidade * ip.preco_unitario) AS subtotal
FROM item_pedido ip
INNER JOIN produto p ON ip.id_produto = p.id_produto;
```

c) Monte o caminho PEDIDO, ITEM_PEDIDO e PRODUTO:

```
SELECT
    ped.id_pedido,
    ped.data_pedido,
    prod.nome AS nome_produto,
    item.quantidade,
    item.preco_unitario,
    (item.quantidade * item.preco_unitario) AS subtotal
FROM pedido ped
INNER JOIN item_pedido item ON ped.id_pedido = item.id_pedido
INNER JOIN produto prod ON item.id_produto = prod.id_produto;
```

d) Liste empregados e respectivos supervisores:

```
SELECT
    e.nome AS empregado,
    s.nome AS supervisor
FROM empregado e
INNER JOIN empregado s ON e.id_supervisor = s.id_empregado;
```

e) Explique por que cada condição ON preserva a semântica:

Explicação: A cláusula ON estabelece a regra precisa de integridade referencial entre a chave primária de uma tabela e a chave estrangeira da tabela relacionada. Ao especificar explicitamente a igualdade lógica (ex.: `ON p.id\_cliente = c.id\_cliente`), garante-se que apenas os registros logicamente vinculados sejam associados no produto cartesiano. Isso evita a mistura inadvertida de dados de entidades distintas e preserva exatamente o significado de negócio do relacionamento modelado no diagrama de banco de dados.

2- Exercício Proposto 2: Junções externas e ausência de correspondência

a) Liste todos os clientes, inclusive os que não possuem pedidos:

```
SELECT
    c.id_cliente,
    c.nome,
    p.id_pedido,
    p.data_pedido
FROM cliente c
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente;
```

b) Localize produtos nunca vendidos:

```
SELECT
    p.id_produto,
    p.nome
FROM produto p
```

```
LEFT JOIN item_pedido ip ON p.id_produto = ip.id_produto
WHERE ip.id_produto IS NULL;
```

c) Reescreva um RIGHT JOIN equivalente com LEFT JOIN:

-- Consulta original com RIGHT JOIN:

```
SELECT c.nome, p.id_pedido
FROM pedido p
RIGHT JOIN cliente c ON p.id_cliente = c.id_cliente;
```

-- Consulta equivalente reescrita com LEFT JOIN:

```
SELECT c.nome, p.id_pedido
FROM cliente c
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente;
```

d) Simule uma junção externa completa em duas tabelas:

```
SELECT c.id_cliente, c.nome, p.id_pedido, p.data_pedido
FROM cliente c
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente
```

UNION

```
SELECT c.id_cliente, c.nome, p.id_pedido, p.data_pedido
FROM cliente c
RIGHT JOIN pedido p ON c.id_cliente = p.id_cliente;
```

e) Explique por que um filtro no WHERE pode eliminar linhas preservadas:

Explicação: Quando realizamos um LEFT JOIN, as linhas da tabela à esquerda sem correspondência na tabela da direita são mantidas na consulta, preenchendo as colunas da tabela direita com valores NULL. No entanto, se adicionarmos uma cláusula `WHERE` referenciando uma coluna da tabela da direita com um filtro comum (exemplo: `WHERE p.status = 'Concluído'`), essa condição será avaliada como FALSO ou UNKNOWN para as linhas preservadas (pois `NULL = 'Concluído'` é desconhecido). Consequentemente, o `WHERE`

descarta todas as linhas preservadas que continham NULLs, transformando involuntariamente a junção externa em um INNER JOIN ostensivo.

3- Exercício Proposto 3: Agregações GROUP BY e HAVING

a) Calcule quantidade, menor, maior e média de preços por categoria:

```
SELECT
    categoria,
    COUNT(*) AS total_produtos,
    MIN(preco) AS menor_preco,
    MAX(preco) AS maior_preco,
    AVG(preco) AS preco_medio
FROM produto
GROUP BY categoria;
```

b) Conte pedidos por cliente, incluindo zero:

```
SELECT
    c.id_cliente,
    c.nome,
    COUNT(p.id_pedido) AS total_pedidos
FROM cliente c
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome;
```

c) Mostre apenas clientes com pelo menos dois pedidos:

```
SELECT
    c.id_cliente,
    c.nome,
    COUNT(p.id_pedido) AS total_pedidos
FROM cliente c
INNER JOIN pedido p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) >= 2;
```

d) Calcule valor total por pedido usando ITEM_PEDIDO:

```
SELECT
    id_pedido,
    SUM(quantidade * preco_unitario) AS valor_total_pedido
FROM item_pedido
GROUP BY id_pedido;
```

e) Justifique cada uso de WHERE, GROUP BY e HAVING:

Justificativa das Cláusulas:

- WHERE: Utilizado para filtrar linhas individuais ANTES da realização do agrupamento e dos cálculos de agregação.
- GROUP BY: Utilizado para agrupar as linhas que possuem valores idênticos nas colunas especificadas, criando subconjuntos para a execução de funções agregadas (COUNT, SUM, AVG, etc.).
- HAVING: Utilizado para filtrar os grupos já resultantes da agregação DEPOIS da execução do GROUP BY, permitindo impor condições sobre resultados de funções agregadas (ex.: `HAVING COUNT(\*) >= 2`).

4- Desafio Integrador: Entregue oito consultas multitabelas e indicadores

a) Consulta 1 (INNER JOIN com qualificação completa):

- Pergunta: Quais são os pedidos e seus respectivos clientes utilizando qualificação completa de tabelas?
- SQL:

```
SELECT cliente.nome AS nome_cliente, pedido.id_pedido, pedido.data_pedido
FROM cliente
INNER JOIN pedido ON cliente.id_cliente = pedido.id_cliente;
```

- Expectativa / Resultado Esperado: Ana Silva, Pedido 101, 2026-09-01 | Bruno Lima, Pedido 102, 2026-09-02
- Interpretação: Garante precisão ao qualificar explicitamente todas as colunas com os nomes das tabelas de origem.

b) Consulta 2 (INNER JOIN de 3 tabelas com qualificação completa):

- Pergunta: Quais produtos compõem cada pedido com seus detalhes de quantidade e preço?
- SQL:

```
SELECT pedido.id_pedido, produto.nome AS nome_produto,  
item_pedido.quantidade, item_pedido.preco_unitario  
FROM pedido  
INNER JOIN item_pedido ON pedido.id_pedido = item_pedido.id_pedido  
INNER JOIN produto ON item_pedido.id_produto = produto.id_produto;
```

- Expectativa / Resultado Esperado: Pedido 101, Notebook, Qtd: 1, Preço: 3500.00 | Pedido 101, Mouse, Qtd: 2, Preço: 50.00
- Interpretação: Navega pelo relacionamento N:M conectando pedido, item_pedido e produto com qualificação total.

c) Consulta 3 (Consulta Externa - Todos os Clientes e Pedidos):

- Pergunta: Qual é a relação geral de todos os clientes e seus pedidos existentes?
- SQL:

```
SELECT c.id_cliente, c.nome, p.id_pedido, p.status  
FROM cliente c  
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente;
```

- Expectativa / Resultado Esperado: Ana Silva - Pedido 101 (Concluído) | Carlos Souza - NULL (Sem Pedidos)
- Interpretação: Mantém o cadastro completo de clientes no relatório principal independentemente do histórico de compras.

d) Consulta 4 (Consulta Externa - Busca por Ausência):

- Pergunta: Quais clientes estão cadastrados no sistema mas nunca realizaram nenhum pedido?
- SQL:

```
SELECT c.id_cliente, c.nome, c.email  
FROM cliente c
```

```
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente  
WHERE p.id_pedido IS NULL;
```

- Expectativa / Resultado Esperado: Carlos Souza, carlao@email.com | Mariane Dias, mari@email.com
- Interpretação: Filtra o resultado do LEFT JOIN utilizando `WHERE p.id\_pedido IS NULL` para encontrar registros sem correspondência.

e) Consulta 5 (Autorrelacionamento):

- Pergunta: Quem é o supervisor direto de cada empregado da empresa?
- SQL:

```
SELECT  
    emp.nome AS nome_empregado,  
    emp.cargo,  
    sup.nome AS nome_supervisor  
FROM empregado emp  
INNER JOIN empregado sup ON emp.id_supervisor = sup.id_empregado;
```

- Expectativa / Resultado Esperado: João Silva (Analista) -> Supervisor: Roberto Santos | Maria Lima (Desenvolvedora) -> Supervisor: Roberto Santos
- Interpretação: Conecta a tabela `empregado` a si mesma através do vínculo entre `id\_supervisor` e `id\_empregado`.

f) Consulta 6 (Consulta Agregada com LEFT JOIN e contagem correta de zero - Clientes):

- Pergunta: Quantos pedidos foram realizados por cada cliente, contabilizando zero para os inativos?
- SQL:

```
SELECT  
    c.id_cliente,  
    c.nome,  
    COUNT(p.id_pedido) AS total_pedidos  
FROM cliente c  
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente  
GROUP BY c.id_cliente, c.nome;
```

- Expectativa / Resultado Esperado: Ana Silva: 3 pedidos | Bruno Lima: 1 pedido | Carlos Souza: 0 pedidos
- Interpretação: Utiliza `COUNT(p.id\_pedido)` em vez de `COUNT(\*)` para garantir a contagem 0 nos clientes sem pedidos.

g) Consulta 7 (Consulta Agregada com LEFT JOIN e contagem correta de zero - Produtos):

- Pergunta: Quantas vezes cada produto foi incluído em itens de pedido, indicando zero para os sem vendas?
- SQL:

```
SELECT
    p.id_produto,
    p.nome,
    COUNT(ip.id_pedido) AS vezes_vendido
FROM produto p
LEFT JOIN item_pedido ip ON p.id_produto = ip.id_produto
GROUP BY p.id_produto, p.nome;
```

- Expectativa / Resultado Esperado: Teclado Mecânico: 12 vezes | Cadeira Gamer: 0 vezes
- Interpretação: O uso de `COUNT(ip.id\_pedido)` sob um LEFT JOIN retorna 0 para itens não associados a nenhum pedido.

h) Consulta 8 (Combinação de WHERE e HAVING):

- Pergunta: Quais categorias de produtos em estoque possuem um preço médio superior a R\$ 100,00 e quantos itens elas possuem?
- SQL:

```
SELECT
    p.categoria,
    COUNT(*) AS qtd_produtos,
    AVG(p.preco) AS preco_medio
FROM produto p
WHERE p.situacao_estoque = 'Em Estoque'
```

GROUP BY p.categoria
HAVING AVG(p.preco) > 100.00;

- Expectativa / Resultado Esperado: Eletrônicos: 5 produtos, Média R\$ 450.00
| Informática: 8 produtos, Média R\$ 220.00
- Interpretação: Aplica `WHERE` para pré-filtrar itens em estoque e `HAVING` para filtrar os grupos cuja média seja maior que 100.